Trading Research Platform

Complete Technical Documentation

SIGNAL GENERATION · BACKTESTING ENGINE · PERFORMANCE METRICS · RISK MANAGEMENT

This document explains every component of the platform in precise detail: how each trading strategy generates signals, how look-ahead bias is prevented, how the backtest engine simulates trades bar by bar, how position sizes are calculated using volatility, how risk is managed with stop-losses and take-profits, and what every performance metric measures.

Table of Contents

1 Architecture Overview

The platform is structured as five independent Python modules that communicate through well-defined interfaces. Each module has a single responsibility. There are three ways to use the platform: the **Trend/Momentum Builder** (build a strategy from one of 12 signals with configurable entry, exit, and filters), the **Mean Reversion Builder** (6 mean-reversion signals with their own entry/exit options), and the **Classic Multi-Strategy Mode** (select and combine multiple pre-configured strategies).

Module Responsibilities

Module	File	Responsibility
Data	`data.py`	Downloads OHLCV price data from Yahoo Finance via `yfinance` and caches it in the Streamlit session.
Strategies	`strategies.py`	Implements all 18 signal generators across both builders, plus signal combination, entry/exit logic, and all filter functions.
Backtest	`backtest.py`	Bar-by-bar simulation engine: converts signals to positions, applies SL/TP, computes returns and all metrics.
Utils	`utils.py`	Shared helpers: return calculation, annualisation factor, all Plotly charting functions.
App	`app.py`	Streamlit UI entry point. Wires all modules together into a three-tab web application.

Data Flow

- `data.py` fetches OHLCV price bars from Yahoo Finance.
- `strategies.py` reads the price DataFrame and produces a signal Series (1, −1, or 0 per bar).
- `backtest.py` reads both the signal Series and the price DataFrame, simulates trades bar by bar, and returns metrics, an equity curve, and a trade log.
- `utils.py` converts those outputs into Plotly charts.
- `app.py` orchestrates the above in order, stores results in Streamlit session state between re-renders, and displays everything across three tabs.

2 Data Ingestion

All market data is sourced from Yahoo Finance through the `yfinance` library. The function `fetch_data(ticker, interval, start, end)` in `data.py` handles downloading, cleaning, and caching.

Supported Timeframes

Interval	Bar Duration	Yahoo Finance Limit	Approx. Bars per Year
`1d`	1 calendar day	No limit	252
`4h`	4 hours	730 days	~411
`1h`	1 hour	730 days	~1,638
`15m`	15 minutes	60 days	~1,638
`5m`	5 minutes	60 days	~4,914

auto_adjust = True

All downloads use `auto_adjust=True`. This instructs Yahoo Finance to return prices already adjusted for historical stock splits and dividend distributions. Without adjustment, a 2:1 stock split would appear in the price history as a 50% crash overnight — the backtest engine would then generate a massive sell signal and record a large loss that never actually happened. Adjusted prices ensure the entire price history is internally consistent and suitable for accurate backtesting.

Session Caching

Downloaded data is stored in `st.session_state` using a cache key of the form `ticker_interval_start_end`. If the user clicks *Run Backtest* again with identical settings, the cached DataFrame is returned immediately without a second network request. The cache is invalidated whenever the ticker, interval, or date range changes.

Data Cleaning

- If `yfinance` returns a MultiIndex column structure (which can occur on some versions), it is flattened to standard column names.
- Only the five standard OHLCV columns are retained: `Open`, `High`, `Low`, `Close`, `Volume`.
- Any row where `Close` is `NaN` is dropped — this can occur at the end of the data range when the most recent bar has not yet closed.
- A minimum of 100 bars is required before a backtest can run. Below this threshold the platform rejects the data with an error message.

3 Signal Convention & Look-Ahead Bias Prevention

All signal generators in the platform — regardless of which builder or strategy type — output a **pandas Series** with the same index as the price DataFrame. Each value in the Series is one of three integers:

Value	Meaning
1	Long signal — go long (buy) at the next bar's open.
−1	Short signal — go short (sell) at the next bar's open.
0	No signal — do nothing, stay flat.

State-Based vs Event-Based Signals

The platform uses two signal types internally. **State-based signals** output ±1 on every bar where the condition holds (e.g. price is above its moving average). **Event-based signals** fire ±1 on exactly one bar — the bar where the condition transitions — and output 0 on all subsequent bars even if the condition persists. The Trend/Momentum Builder always converts signals to state-based internally before applying confirmation logic.

The One-Bar Delay Rule

Look-ahead bias is prevented by applying a one-bar shift to all signals inside the backtest engine:

```
shifted_signals = signals.shift(1).reindex(prices.index).fillna(0)
```

This means: a signal generated at the close of bar N is only acted upon at bar N+1. The position is entered at the close of bar N (used as the entry price proxy), and the trade result is calculated from bar N+1 onwards. This correctly models the real-world workflow: observe the close, decide to trade, execute at or near the next open.

Every indicator used in every signal (moving averages, RSI, MACD, ATR, ADX, Bollinger Bands, etc.) is computed using only data up to and including bar N. No future price information is used in any calculation. The `.shift(1)` operation is the single enforcement point — without it, any signal that reads a close price at bar N and then trades at bar N's close would be using bar N's close to determine whether to enter a trade at bar N's close, which is impossible in live trading.

4 Trend/Momentum Builder

The Trend/Momentum Builder allows you to construct a strategy from any of 12 signal generators. Each signal captures a different aspect of price momentum or trend direction. You configure entry logic, exit logic, and optional filters independently of the signal choice.

4.1 Signal Types

Quick reference — all 12 signals, their type, configurable parameters, and defaults:

Signal	Type	Key Parameters	Default
TSMOM	Momentum	lookback, threshold	252 bars, 0.0
SMA Crossover	Trend	fast_period, slow_period	10, 50
EMA Crossover	Trend	fast_period, slow_period	10, 50
Price vs MA	Trend	period, ma_type, buffer	50, SMA, 0.0
MA Slope	Trend	period, ma_type, slope_threshold	20, SMA, 0.0
MACD Crossover	Momentum	fast, slow, signal_period	12, 26, 9
MACD Histogram	Momentum	fast, slow, signal_period	12, 26, 9
Donchian Channels	Breakout	period, buffer	20, 0.0
Turtle Breakout	Breakout	entry_period, exit_period	20, 10
ADX + DI	Trend	period, threshold	14, 25.0
SuperTrend	Trend	period, multiplier	10, 3.0
Parabolic SAR	Trend	step, max_step	0.02, 0.2

TSMOM — Time-Series Momentum

Type: Momentum | Parameters: lookback (default 252), threshold (default 0.0)

Time-Series Momentum is one of the most robust and academically documented signals in systematic trading. It measures whether an asset's own recent return is positive or negative. A positive N-bar return implies upward momentum; a negative return implies downward momentum.

How it is calculated:

```
Return[t] = (Close[t] - Close[t - lookback]) / Close[t - lookback]

Long  (+1): Return[t] > threshold
Short (-1): Return[t] < -threshold
```

With `threshold=0.0` (default), the signal is long whenever the lookback-bar return is positive, and short when negative. A positive threshold requires a stronger move before a signal fires, reducing noise at the cost of entering trends later.

Why it works: Markets exhibit momentum — assets that have performed well recently tend to continue outperforming over intermediate horizons (1–12 months). This is one of the most persistent anomalies documented in academic finance.

Practical considerations:

- The default lookback of 252 bars (one year on daily data) captures intermediate-term momentum. Shorter lookbacks (60–120 bars) capture faster momentum but with more noise.
- On daily data, TSMOM flips direction whenever the 1-year return crosses zero — this produces relatively few trades per year.
- TSMOM is a state-based signal: it holds +1 or −1 continuously. There is no neutral zone unless a positive threshold is set.

SMA Crossover

Type: Trend-following | Parameters: fast_period (default 10), slow_period (default 50)

The Simple Moving Average Crossover generates signals when a shorter-period SMA crosses above or below a longer-period SMA. It is one of the oldest and most widely used trend-following techniques.

```
SMA[N][t] = mean(Close[t-N+1 : t+1])   # arithmetic mean of last N closes

Long  (+1): fast[t] > slow[t]  AND  fast[t-1] <= slow[t-1]  (fast crosses above slow)
Short (-1): fast[t] < slow[t]  AND  fast[t-1] >= slow[t-1]  (fast crosses below slow)
```

The crossover is an **event-based signal**: it fires a +1 or −1 only on the bar of the crossover itself, not on every subsequent bar where the fast MA is above the slow MA. The backtest engine forward-fills this into a held position (holding until the opposite crossover).

Practical considerations:

- `fast_period` must be strictly less than `slow_period`.
- Shorter fast periods (e.g. 5/20) generate more signals and enter trends earlier, but also produce more whipsaws in sideways markets.
- Longer combinations (e.g. 50/200 — the "Golden Cross") generate fewer signals and are more robust in trending markets, but enter trends late and exit late.
- SMA crossovers lag by nature — they cannot signal the exact turning point. Their value comes from riding the middle portion of sustained trends.

EMA Crossover

Type: Trend-following | Parameters: fast_period (default 10), slow_period (default 50)

Like SMA Crossover but uses Exponential Moving Averages — each bar's EMA gives progressively more weight to recent prices. The EMA reacts faster to price changes than the SMA.

```
EMA multiplier  k = 2 / (N + 1)
EMA[t] = Close[t] * k  +  EMA[t-1] * (1 - k)

Long  (+1): fast EMA crosses above slow EMA
Short (-1): fast EMA crosses below slow EMA
```

Signal logic is identical to SMA Crossover — long when fast EMA crosses above slow EMA, short when it crosses below. Because EMAs react more quickly, crossovers occur sooner than with SMAs, generating more trades and entering trends earlier but with more whipsaws.

Practical considerations:

- `fast_period` must be strictly less than `slow_period`.
- EMA crossovers are more sensitive to recent price spikes than SMA crossovers — a single large bar can cause a crossover that reverses within a few bars.
- For the same parameter values, EMA crossovers will typically generate more trades than SMA crossovers.

Price vs MA

Type: Trend | Parameters: period (default 50), ma_type (SMA/EMA, default SMA), buffer (default 0.0)

A state-based signal: long while price is above its moving average, short while below. Unlike crossover signals, this holds ±1 continuously — it does not require a transition to fire.

```
Long  (+1): Close[t] > MA[t] * (1 + buffer)
Short (-1): Close[t] < MA[t] * (1 - buffer)
```

The `buffer` parameter creates a neutral zone around the MA. With `buffer=0.02` (2%), price must move 2% above the MA to go long, and 2% below to go short. This prevents frequent signal flips when price is oscillating around the average.

Practical considerations:

- This is the simplest trend filter available. It generates a signal on every bar and produces many trades.
- The buffer is important for reducing noise — without it, price crossing the MA repeatedly over a few bars generates many rapid reversals.
- A 200-period SMA version of this signal is functionally equivalent to the classic "price above/below 200-day MA" regime filter.

MA Slope

Type: Trend | Parameters: period (default 20), ma_type (SMA/EMA, default SMA), slope_threshold (default 0.0)

Rather than comparing price to its MA, this signal measures whether the MA itself is rising or falling.

```
Slope[t] = MA[t] - MA[t-1]

Long  (+1): Slope[t] > slope_threshold
Short (-1): Slope[t] < -slope_threshold
```

A rising MA indicates a prevailing uptrend; a falling MA indicates a downtrend. This is smoother than Price vs MA because it ignores short-term price spikes that temporarily cross the average while the trend remains intact.

Practical considerations:

- MA Slope with period=20 is quite responsive and will flip frequently in choppy markets.
- The `slope_threshold` parameter can filter out near-flat MAs — only trading when the MA is sloping with meaningful conviction.
- For raw slope values, the threshold must be calibrated to the asset's price level and volatility — a slope threshold of 0.5 means something very different for a $5 stock vs a $500 stock. Consider using percentage-normalised slope for cross-asset consistency.

MACD Crossover

Type: Momentum | Parameters: fast (default 12), slow (default 26), signal_period (default 9)

The Moving Average Convergence Divergence (MACD) is a trend-momentum hybrid that measures the relationship between two EMAs. The signal line is an EMA of the MACD line itself.

```
MACD Line   = EMA(Close, fast) - EMA(Close, slow)
Signal Line = EMA(MACD Line, signal_period)
Histogram   = MACD Line - Signal Line

Long  (+1): MACD Line crosses ABOVE Signal Line
Short (-1): MACD Line crosses BELOW Signal Line
```

This is an **event-based signal**: it fires only on the bar of the crossover. The default parameters (12, 26, 9) are the classic MACD settings used across institutional and retail platforms worldwide.

Why it works: The MACD measures the rate of change of a trend. A crossover above the signal line indicates accelerating upward momentum; below indicates accelerating downward momentum. It identifies transitions in trend momentum, not just trend direction.

Practical considerations:

- MACD crossovers lag by the signal period (9 bars by default) — the signal line is a 9-bar EMA of the MACD line, so crossovers occur after the underlying momentum shift.
- MACD performs poorly in ranging, choppy markets where the lines oscillate around zero and produce frequent false crossovers.
- The chart displays the MACD line, signal line, and histogram as a panel below the price chart.

MACD Histogram

Type: Momentum | Parameters: fast (default 12), slow (default 26), signal_period (default 9)

Uses the same MACD calculation but generates a state-based signal from the histogram (MACD Line minus Signal Line) rather than a crossover event.

```
Histogram[t] = MACD Line[t] - Signal Line[t]

Long  (+1): Histogram[t] > 0  (MACD above signal — momentum accelerating upward)
Short (-1): Histogram[t] < 0  (MACD below signal — momentum accelerating downward)
```

Unlike MACD Crossover (which fires only on the bar of the crossover), MACD Histogram holds +1 or −1 for the entire period the histogram is on one side of zero. This produces longer-held positions and trades every flip of the histogram.

Practical considerations:

- The histogram oscillates frequently — MACD Histogram generates more trades than MACD Crossover.
- The chart displays the histogram as green/red bars alongside the MACD and signal lines.
- Because MACD Histogram is state-based, it is equivalent to MACD Crossover but held continuously rather than only on the crossover bar.

Donchian Channels

Type: Breakout | Parameters: period (default 20), buffer (default 0.0)

Donchian Channels define a price envelope using the highest high and lowest low over a rolling window. A breakout above the upper channel signals strength; a breakout below the lower channel signals weakness.

```
Upper Channel[t] = max(High[t-period : t])   # highest high over last 'period' bars
Lower Channel[t] = min(Low[t-period  : t])   # lowest low  over last 'period' bars

Long  (+1): Close[t] > Upper Channel[t-1] * (1 + buffer)   # breakout above upper
Short (-1): Close[t] < Lower Channel[t-1] * (1 - buffer)   # breakdown below lower
```

Signals fire only on the **first bar** of the breakout (event-based). Using the previous bar's channel (`t-1`) is critical to prevent look-ahead bias — the channel boundary is fully known at bar N−1's close before bar N opens.

The `buffer` parameter requires the breakout to exceed the channel by a percentage margin, filtering out marginal breakouts that barely clip the channel boundary.

Practical considerations:

- A 20-bar Donchian Channel (one month on daily data) is a classic trend-following parameter.
- Donchian signals can go through extended drawdowns in choppy markets where breakouts fail repeatedly.
- The channel boundaries are displayed on the price chart as a shaded envelope.

Turtle Breakout

Type: Breakout | Parameters: entry_period (default 20), exit_period (default 10)

The Turtle Trading system, made famous by Richard Dennis in the 1980s, uses two Donchian channels of different lengths — a longer one for entry and a shorter one for exit.

```
Entry Long:  Close > Highest High over last entry_period bars
Entry Short: Close < Lowest Low  over last entry_period bars

Exit Long:   Close < Lowest Low  over last exit_period bars
Exit Short:  Close > Highest High over last exit_period bars
```

This is a **state-based signal**: it holds +1 after a long entry until the exit condition is met, and −1 after a short entry until the exit condition is met. The exit period is typically half the entry period.

Why it differs from Donchian: Donchian uses a single channel — it exits when an opposite breakout fires. Turtle uses a separate, shorter channel for exits, meaning it exits a long trade when price breaks a 10-bar low, not when it breaks a 20-bar high in the other direction. This produces earlier exits and preserves more profit.

Practical considerations:

- `entry_period` must be greater than `exit_period`.
- The 20/10 default is the classic Turtle configuration.
- The original Turtle system also used a 55/20 combination as a secondary (more aggressive) entry system.

ADX + DI

Type: Trend + Crossover | Parameters: period (default 14), threshold (default 25.0)

The Average Directional Index (ADX) combined with Directional Indicators (+DI and −DI) identifies both the strength of a trend and its direction.

```
+DI[t] = 100 * EMA(+DM, period) / ATR(period)
-DI[t] = 100 * EMA(-DM, period) / ATR(period)
ADX[t] = 100 * EMA(|+DI - -DI| / (+DI + -DI), period)

Long  (+1): ADX[t] > threshold  AND  +DI crosses ABOVE -DI
Short (-1): ADX[t] > threshold  AND  -DI crosses ABOVE +DI
Flat  ( 0): ADX[t] < threshold  (insufficient trend strength)
```

The ADX acts as a gate: signals only fire when the market is trending above the threshold. This prevents the DI crossovers from generating false signals during flat, ranging conditions.

Practical considerations:

- ADX values below 20–25 indicate a ranging or flat market. Between 25–50 indicates a moderate trend. Above 50 indicates a strong trend.
- ADX lags — it confirms trends after they have begun, not at their start.
- The threshold of 25 is widely cited as the boundary between ranging and trending. Increasing it to 30–35 reduces the number of signals but increases their quality.
- The chart displays ADX and ±DI as a panel below the price chart.

SuperTrend

Type: Trend | Parameters: period (default 10), multiplier (default 3.0)

SuperTrend is an ATR-based trailing band indicator that produces a single support/resistance line that flips side when price crosses it.

```
Step 1: ATR = Average True Range over period bars

Step 2: Upper Band = (High + Low) / 2  +  multiplier * ATR
        Lower Band = (High + Low) / 2  -  multiplier * ATR

Step 3: The SuperTrend line tracks whichever band is active.
        Starts on the Lower Band (bullish).
        Switches to Upper Band (bearish) when Close crosses below Lower Band.
        Switches back to Lower Band when Close crosses above Upper Band.

Long  (+1): Close > SuperTrend line  (price above the trailing support)
Short (-1): Close < SuperTrend line  (price below the trailing resistance)
```

This is a **state-based signal** — it holds ±1 until price crosses the SuperTrend line.

Why it works: The ATR-based band automatically widens in volatile conditions, preventing premature stop-outs during normal volatility. The multiplier controls how far the band sits from price — higher values mean fewer flips and longer-held positions.

Practical considerations:

- SuperTrend is displayed on the price chart as a green (bullish) or red (bearish) line beneath/above price.
- The period and multiplier together determine sensitivity — `period=7, multiplier=3` is more reactive; `period=14, multiplier=3` is smoother.
- SuperTrend can produce extended flat periods on highly volatile assets where the band keeps getting crossed back and forth.

Parabolic SAR

Type: Trend | Parameters: step (default 0.02), max_step (default 0.2)

The Parabolic Stop and Reverse (PSAR) system, developed by J. Welles Wilder, uses an accelerating trailing stop that tightens as a trade becomes more profitable.

```
AF starts at 'step' and increases by 'step' each bar the extreme point extends,
up to a maximum of 'max_step'.

SAR[t] = SAR[t-1]  +  AF * (Extreme Point - SAR[t-1])

Long  (+1): Close > SAR  (price above the trailing dot)
Short (-1): Close < SAR  (price below the trailing dot)
```

When price crosses the SAR, the system reverses: a long becomes a short (or vice versa). This is a **state-based signal** — it is always either long or short, never flat.

Why it works: The accelerating factor means the SAR tightens as a trade runs further in your favour — locking in profits as the move extends. Early in a trade the SAR is loose (allows the position to breathe); later it tightens.

Practical considerations:

- Lower step values produce a slower-moving SAR that stays in trends longer but gives back more profit on reversals.
- Higher step values produce a faster SAR that exits sooner but also generates more false reversals in choppy markets.
- The PSAR dots are displayed on the price chart.
- Because PSAR is always long or short, it will trade continuously — there are no flat (neutral) periods unless filtered externally.

4.2 Entry Logic

Confirmation Bars

By default (slider = 1), a trade opens immediately on the first bar the signal fires. Setting confirmation bars to N requires **N consecutive bars** all showing the same signal direction before the trade opens.

Example: with `confirmation_bars = 3`, if the signal reads +1 on bars 10, 11, 12, the trade opens at the close of bar 12. If the signal reads +1, +1, −1, no trade opens — the streak resets.

The confirmation bars slider ranges from 1 (enter immediately) to 10. This is most useful for noisier signals (crossovers, Donchian) where a multi-bar confirmation filters out one-bar false breakouts.

> **Note:** Confirmation is applied after the signal has been forward-filled into state form, so it works correctly for both state-based and event-based signals.

4.3 Exit Logic

Two exit rules are available:

Signal Reversal (default)

The position closes when the signal flips to the opposite direction. A long trade closes when the signal becomes −1; a short trade closes when the signal becomes +1. This is the most natural exit for state-based trend signals — you stay in the trade as long as the trend persists.

Fixed Bars Held

The position is force-closed after a fixed number of bars regardless of the signal. The slider ranges from 1 to 200 bars (default 20). The trade closes at the close of bar (entry + max_bars). If the signal reverses before then, the reversal exit takes precedence.

> **Note:** SL/TP (configured in the Exit Logic section) operates independently of and alongside these rules. A trade can be exited by its SL or TP even before the signal exit condition is met.

4.4 Filters

Three optional filters are available. Each filter zeroes out signal bars that do not pass the condition — the trade simply does not open on those bars.

Long Only / Short Only

Directional constraints. **Long only** means all short (−1) signals are ignored. **Short only** means all long (+1) signals are ignored. Checking both simultaneously produces no trades.

ADX Strength Filter

ADX (Average Directional Index) measures how strongly trending a market is on a scale of 0–100, regardless of direction. When enabled, any signal bar where ADX is below the minimum threshold is zeroed out — only trades during confirmed trending conditions.

```
Signal[t] = 0  if ADX[t] < adx_min
```

Values below 20–25 indicate a ranging or choppy market. The default threshold is 25. This filter is most effective for crossover strategies that generate many false signals in flat markets.

Higher-TF Trend Filter

Resamples the existing price data to a higher timeframe, computes a moving average on that higher-TF data, and uses it as a directional gate. Long signals are blocked when the higher-TF trend is bearish; short signals are blocked when it is bullish.

Trading Interval	Higher TF Used
5 minutes	1 hour
15 minutes	4 hours
1 hour	1 day (daily)
4 hours	1 day (daily)
1 day	1 week (weekly)

The MA period for the higher-TF trend is configurable (default 50 bars on the higher TF). This filter converts any strategy into a regime-aware strategy — only trading in the direction of the bigger picture.

4.5 Stop-Loss & Take-Profit

SL/TP is disabled by default. When enabled, volatility-based stops are calculated at trade entry. See Section 7.2 for the full calculation methodology. The TP ratio slider (default 2.0) sets the reward-to-risk ratio — a TP ratio of 2.0 means the take-profit is placed at twice the stop-loss distance from entry.

A **trailing stop** option is also available. When enabled, the stop-loss level ratchets up (for longs) or down (for shorts) as the trade moves in your favour, using an ATR-based calculation.

5 Mean Reversion Builder

Mean reversion strategies exploit the tendency of prices to return to their average after temporary deviations. They perform best in range-bound, low-trend markets and tend to struggle during sustained directional trends.

5.1 Signal Types

Quick reference — all 6 mean-reversion signals:

Signal	Type	Key Parameters	Default
Z-Score	State	lookback, entry_threshold, exit_threshold	20, 2.0, 0.5
Bollinger Bands	Event	period, std_dev	20, 2.0
RSI	Event	period, oversold, overbought	14, 30, 70
Short-Term Reversal	State	lookback, threshold	5, 2.0%
MA Deviation	State	period, ma_type, threshold	20, SMA, 2.0%
Range Reversion	State	lookback, threshold	20, 20%

Z-Score

Type: Mean-reversion (state) | Parameters: lookback (default 20), entry_threshold (default 2.0), exit_threshold (default 0.5)

The Z-Score measures how many standard deviations the current price is from its rolling mean. It produces a state-based signal with built-in entry and exit thresholds.

```
z[t] = (Close[t] - mean(Close, lookback)) / std(Close, lookback)

Long  (+1):  z[t] < -entry_threshold    (price more than 2σ below mean)
Exit long:   z[t] > -exit_threshold     (price has reverted past -0.5σ)

Short (-1):  z[t] > +entry_threshold    (price more than 2σ above mean)
Exit short:  z[t] < +exit_threshold     (price has reverted past +0.5σ)
```

This is the most complete MR signal — both entry and exit thresholds are configurable. The `exit_threshold` determines how far the reversion must travel before the trade closes. `exit_threshold=0` means exit at the mean; `exit_threshold=0.5` (default) means exit when price has reverted to within 0.5σ of the mean (partial reversion capture).

Practical considerations:

- Shorter lookbacks (10–15) make the Z-Score more reactive to recent moves, generating more signals.

- The entry threshold controls selectivity — higher values (e.g. 2.5–3.0) produce fewer but higher-conviction entries.
- The Z-Score assumes approximate stationarity over the lookback window. If price is in a persistent trend, the mean shifts and extreme Z-Score values may not revert.

Bollinger Bands

Type: Mean-reversion (event) | Parameters: period (default 20), std_dev (default 2.0)

Bollinger Bands define a volatility envelope around a moving average. The upper and lower bands are placed at ±`std_dev` standard deviations from the MA. When price touches or crosses a band, it is considered statistically extreme and likely to revert.

```
Middle Band = SMA(Close, period)
Upper Band  = Middle Band + std_dev * std(Close, period)
Lower Band  = Middle Band - std_dev * std(Close, period)

Long  (+1): Close crosses DOWN through Lower Band  (price enters oversold zone)
Short (-1): Close crosses UP through Upper Band    (price enters overbought zone)
```

Signals fire only on the **bar price first crosses the band** (event-based). This is a contrarian entry — buying when price is falling below the lower band, shorting when rising above the upper band.

Practical considerations:

- Bollinger Bands adapt to volatility — in high-volatility periods the bands widen, requiring a larger move to trigger a signal; in low-volatility periods the bands narrow, making signals more frequent.
- The default 2.0 std_dev setting means approximately 95% of price action stays within the bands under normal conditions.
- In trending markets, price can "walk the band" — repeatedly triggering signals without reverting. The ADX/ranging filter helps avoid this.

RSI

Type: Mean-reversion (event) | Parameters: period (default 14), oversold (default 30), overbought (default 70)

The Relative Strength Index (RSI) measures the speed and change of price movements, normalised to a 0–100 scale. Values below the oversold threshold indicate the asset is potentially oversold; values above the overbought threshold indicate potential overbuying.

```
RS    = Average Gain over period / Average Loss over period
RSI   = 100 - (100 / (1 + RS))

Long  (+1): RSI crosses back ABOVE oversold level  (RSI was below, now rising above)
Short (-1): RSI crosses back BELOW overbought level (RSI was above, now falling below)
```

Important: The signal fires on **reversal confirmation** — it enters when RSI *exits* the oversold/overbought zone, not when it enters it. This ensures the extreme reading is abating, not deepening.

Practical considerations:

- The 30/70 defaults are conventional. Tighter bands (40/60) generate more signals; wider bands (20/80) generate fewer but more extreme entries.
- In strong trending markets, RSI can remain in oversold/overbought territory for extended periods without reverting. The RSI signal is not suitable as a standalone strategy in trending markets.
- The chart displays RSI as a panel below the price chart with horizontal lines at the oversold and overbought levels.

Short-Term Reversal

Type: Mean-reversion (state) | Parameters: lookback (default 5), threshold (default 2.0%)

Short-Term Reversal fades recent price moves — it bets that large short-term moves will partially retrace. This is a contrarian signal based on the short-term overreaction effect.

```
Return[t] = (Close[t] - Close[t - lookback]) / Close[t - lookback] * 100

Long  (+1): Return[t] < -threshold   (price has fallen more than threshold% — bet on bounce)
Short (-1): Return[t] > +threshold   (price has risen more than threshold% — bet on pullback)
```

The signal holds ±1 while the condition persists. As price reverts, the lookback return shrinks back toward zero and the signal turns off.

Practical considerations:

- This works at short horizons (1–10 bars) where momentum is often negative. At longer horizons, momentum tends to be positive (the TSMOM effect).
- The threshold prevents trading on every minor fluctuation.
- Short-Term Reversal is most effective in liquid, large-cap assets where temporary overreactions are more likely to revert quickly.

MA Deviation

Type: Mean-reversion (state) | Parameters: period (default 20), ma_type (SMA/EMA, default SMA), threshold (default 2.0%)

MA Deviation measures how far price has stretched from its moving average as a percentage.

```
Deviation[t] = (Close[t] - MA[t]) / MA[t] * 100

Long  (+1): Deviation[t] < -threshold   (price more than threshold% below MA)
Short (-1): Deviation[t] > +threshold   (price more than threshold% above MA)
```

This is the percentage-based cousin of Z-Score. Rather than normalising by standard deviation, it uses a fixed percentage threshold. This makes it more intuitive but less adaptive to changing volatility — in a high-volatility regime, the same percentage deviation may be less extreme than usual.

Practical considerations:

- The threshold should be calibrated to the asset's typical daily range. For equities, 2–3% is reasonable; for cryptocurrencies, 5–10% may be needed.

- MA Deviation is simpler to interpret than Z-Score but does not automatically adjust for regime volatility changes.

Range Reversion

Type: Mean-reversion (state) | Parameters: lookback (default 20), threshold (default 20%)

Range Reversion measures where the current price sits within the recent high-low range.

```
Range Position[t] = (Close[t] - Low(lookback)) / (High(lookback) - Low(lookback)) * 100

Long  (+1): Range Position[t] < threshold         (price in the bottom threshold% of range)
Short (-1): Range Position[t] > (100 - threshold) (price in the top threshold% of range)
```

With `threshold=20%`, the signal goes long when price is in the bottom 20% of its 20-bar range, and short when in the top 20%.

Practical considerations:

- This is a purely relative measure — it doesn't depend on absolute price levels or volatility scaling.
- It works best in genuinely range-bound markets where the channel boundaries are somewhat predictable.
- In a trending market, the range shifts progressively and the signal will always be near the extreme of the range, generating continuous signals in the trend direction — the opposite of what is intended.

5.2 Entry Logic

Confirmation Bars

Identical to the Trend/Momentum Builder. Setting this above 1 requires N consecutive bars confirming the signal direction before entry. For state-based MR signals (Z-Score, STR, MA Deviation, Range Reversion), this requires N consecutive bars where the extreme condition persists. For event-based signals (RSI, Bollinger), the signal is first forward-filled before confirmation is applied.

Wait for Indicator to Start Reversing

When this checkbox is enabled, the platform holds off entry until the underlying indicator ticks in the right direction on that bar.

- For a **long signal**: the indicator must be rising (`indicator[today] > indicator[yesterday]`).
- For a **short signal**: the indicator must be falling.

Example with RSI: if RSI is at 25 and still falling to 24, the entry is suppressed. The trade only opens on the first bar RSI ticks upward — providing early evidence that the selling pressure is abating.

This is a one-bar check with no minimum magnitude requirement. It acts as a sanity filter rather than a strong confirmation signal, but reduces the frequency of entering while the move is still running against you.

5.3 Exit Logic

Three exit rules are available for Mean Reversion strategies:

Mean Reversion (default)

The position closes when the signal returns to neutral (0). For state-based signals, this means the extreme condition is no longer active — the price has reverted sufficiently for the signal to turn off. This is the most natural exit for MR: the trade closes once the reversion is complete.

Signal Reversal

The position holds until an opposite entry signal fires — e.g. a long trade stays open until a short signal activates. This produces longer average trade durations than the mean reversion exit.

Fixed Bars Held

Force-closes the position after a fixed number of bars (default 10, range 1–100). This is appropriate when you want to capture short-term mean reversion quickly and limit exposure time regardless of whether full reversion has occurred.

5.4 Filters

Long Only / Short Only

Same as T/M Builder. Directional constraints that block trades in the excluded direction.

Ranging Market Filter

Blocks entries when ADX is **above** the maximum threshold (default 25). The logic is the inverse of the T/M ADX filter — MR strategies need a ranging market, so entries are blocked when ADX indicates a strong trend.

```
Signal[t] = 0  if ADX[t] > adx_max
```

ADX above 25–30 typically indicates the market is trending directionally, making mean reversion dangerous.

Low-Volatility Filter

Blocks entries when annualised volatility exceeds the threshold (default 40%). Volatility is computed as the rolling 20-bar standard deviation of returns, annualised by multiplying by sqrt(252).

```
ann_vol  = std(returns, 20) * sqrt(252)
Signal[t] = 0  if ann_vol[t] > vol_max
```

High-volatility regimes make mean reversion risky — extreme readings can become more extreme before reverting. This filter protects against entering during market dislocations.

5.5 Stop-Loss & Take-Profit

SL/TP is disabled by default for mean reversion strategies. The signal exit threshold (e.g. Z-Score `exit_threshold`) acts as the primary risk control — the trade closes when the reversion is deemed complete. Adding volatility-based SL/TP can cause premature exits before the reversion fully plays out, particularly on assets with high intraday noise. If enabled, the same volatility-based calculation described in Section 7.2 applies.

6 Classic Multi-Strategy Mode

The Classic mode allows multiple pre-configured strategies to be selected simultaneously, with their signals combined into a single composite signal. Each strategy in Classic mode has fixed parameters tuned for its signal type — there are no parameter sliders. The value of Classic mode lies in its signal combination logic.

6.1 Signal Combination

When two or more strategies are selected in Classic mode, their individual signal Series are combined into a composite signal using one of three methods:

Method 1: Majority Vote

The composite signal at each bar is the sign of the sum of individual signals. A long (+1) composite is produced only when more strategies signal long than short; a short (−1) only when more signal short. A tie produces a flat (0) signal.

```
composite[t] = sign( sum(signal_i[t] for all i) )

Example — 3 strategies:
  Strategy A: +1
  Strategy B: +1
  Strategy C: -1
  Sum = +1  →  composite = +1 (majority long)
```

Majority Vote is the most conservative combination method — it requires true consensus before acting. It reduces false signals but also reduces trade frequency compared to any individual strategy.

Method 2: Weighted Average

Each strategy is assigned a weight (configurable, default equal weights). The composite signal is the weighted sum of individual signals, thresholded to produce ±1 or 0.

```
raw_composite[t] = sum( weight_i * signal_i[t] )

composite[t] = +1  if raw_composite[t] > threshold
composite[t] = -1  if raw_composite[t] < -threshold
composite[t] =  0  otherwise
```

Weights are normalised to sum to 1 before the calculation. The threshold (default 0.0) controls how much weighted agreement is needed — a threshold of 0.5 with equal weights among 3 strategies means at least 2 of 3 must agree before a signal fires.

Weighted Average is more flexible than Majority Vote because it allows you to assign higher confidence to certain strategies. A strategy with a weight of 0.5 effectively counts as much as the other two combined.

Method 3: Threshold

A simple threshold on the sum of signals. The composite fires +1 only if the number of long signals ≥ the threshold; −1 only if the number of short signals ≥ the threshold.

```
n_long  = count( signal_i[t] == +1 )
n_short = count( signal_i[t] == -1 )

composite[t] = +1  if n_long  >= threshold
composite[t] = -1  if n_short >= threshold
composite[t] =  0  otherwise
```

With 4 strategies selected and threshold=3, all 4 must agree for a signal (or at least 3 of 4). Setting threshold=1 is equivalent to a union — any strategy signalling is enough to act.

> **Note on signal combination timing:** All signals are computed independently using the one-bar delay rule (Section 3) before combination. The combination logic operates on the already-shifted signals, so there is no additional look-ahead from the combination step itself.

7 Backtest Engine

The backtest engine in `backtest.py` converts a signal Series and a price DataFrame into a complete simulation of trading over the historical period. It operates bar by bar, respecting the one-bar delay rule, and produces a comprehensive set of outputs: an equity curve, a trade log, and all performance metrics.

7.1 Bar-by-Bar Simulation

The simulation iterates through every bar in the price history in chronological order. The order of operations at each bar is strictly defined to prevent any form of look-ahead:

1. **Read the shifted signal** for the current bar (signal from the previous bar's close, already shifted by 1).
2. **Check SL/TP:** If a position is open, check whether the current bar's high or low has touched the stop-loss or take-profit level. If so, record the exit and close the position.
3. **Check exit conditions:** If no SL/TP exit occurred, check whether the signal has reversed or the max bars held has been reached. If so, close the position.
4. **Check entry conditions:** If no position is currently open, check whether the current bar's signal meets the entry criteria (including confirmation bars). If so, open a new position.
5. **Record the bar's return** for the open position (if any).
6. **Advance to the next bar.**

The key variables tracked by the engine at each bar:

Variable	Description
`position`	Current position direction: +1 (long), −1 (short), 0 (flat)
`entry_price`	Close price at which the current position was entered
`entry_bar`	Bar index at which the current position was entered
`sl_price`	Stop-loss price for the current position (None if SL disabled)
`tp_price`	Take-profit price for the current position (None if TP disabled)
`equity`	Running portfolio value, starting at 1.0 (normalised)
`returns`	Series of bar-by-bar returns for the open position
`trade_log`	List of completed trade records (dict per trade)
`pending_entry`	Flag for the pending entry mechanism (see Section 7.6)

SL/TP Hit Detection

The engine checks SL and TP using the bar's high and low rather than just the close. For a long position:

- SL is hit if `Low[t] ≤ sl_price` — the bar touched the stop level.
- TP is hit if `High[t] ≥ tp_price` — the bar touched the take-profit level.
- If **both** are hit on the same bar (a wide-range bar), the stop-loss takes precedence (conservative assumption).

For a short position, the logic is reversed: SL is hit if `High[t] ≥ sl_price`; TP is hit if `Low[t] ≤ tp_price`.

Exit prices for SL/TP are recorded as the SL or TP price itself (not the close), which accurately models execution at the stop or limit level.

7.2 Volatility-Based Stop-Loss Sizing

Motivation

A fixed-percentage stop-loss (e.g. always use a 2% stop) is suboptimal because it ignores the asset's current volatility. A 2% stop on a low-volatility asset might be rarely hit; the same 2% stop on a high-volatility cryptocurrency might be hit on almost every bar just from normal intraday noise.

The platform uses a **volatility-based stop**: the stop distance is set as a multiple of the asset's recent Average True Range (ATR). This ensures the stop is sized relative to how much the asset normally moves — wide enough to survive normal volatility, tight enough to limit losses on genuine reversals.

Calculation

```
ATR[t] = Average True Range over the last atr_period bars
       = mean( max(High[i] - Low[i],
                   |High[i] - Close[i-1]|,
                   |Low[i]  - Close[i-1]|)
               for i in [t-atr_period+1 : t+1] )

Stop Distance = atr_multiplier * ATR[t]

For a LONG entry at price P:
    SL Price = P - Stop Distance
    TP Price = P + Stop Distance * tp_ratio

For a SHORT entry at price P:
    SL Price = P + Stop Distance
    TP Price = P - Stop Distance * tp_ratio
```

The ATR period and multiplier are configurable. Interval-specific default bounds:

Interval	ATR Period	Multiplier Range	Default Multiplier
1d	14	0.5 – 5.0	2.0
4h	14	0.5 – 5.0	2.0
1h	14	0.5 – 5.0	2.0
15m	14	0.5 – 3.0	1.5
5m	14	0.5 – 3.0	1.5

The stop distance is calculated at the bar of entry and **does not change** during the trade's lifetime (unless trailing stop is enabled — see Section 4.5). Even if ATR changes after entry, the stop level remains fixed at the entry-bar ATR value.

7.3 Position Sizing

Two position sizing modes are available:

Mode 1: Fixed Risk Per Trade

Risk a fixed percentage of current equity on each trade. The position size is calculated so that if the stop-loss is hit, the portfolio loses exactly the specified percentage.

```
Risk Amount  = Equity * risk_per_trade_pct / 100
Stop Distance = atr_multiplier * ATR[entry_bar]

Position Size (units) = Risk Amount / Stop Distance
```

Example: equity = $10,000, risk = 1%, ATR-based stop = $5.00 per unit. Position size = $100 / $5.00 = 20 units. If the stop is hit, the loss is exactly 20 × $5.00 = $100 = 1% of equity.

This mode requires SL to be enabled. If SL is disabled, the engine falls back to a fixed fraction of equity (equal-weighted) per trade.

Mode 2: Volatility Targeting

Size positions so that the portfolio's annualised volatility contribution from each trade equals the target. This ensures consistent portfolio-level risk regardless of the individual asset's volatility.

```
ann_vol[t]    = std(returns, vol_lookback) * sqrt(annualisation_factor)
position_size = vol_target / ann_vol[t]
```

When an asset's volatility is high, smaller positions are taken; when low, larger positions. This is the standard approach used by CTA/systematic macro funds.

7.4 Take-Profit Calculation

```
For a LONG trade with entry price P and stop at SL:
    Stop Distance = P - SL
    TP Price      = P + Stop Distance * tp_ratio

For a SHORT trade with entry price P and stop at SL:
    Stop Distance = SL - P
    TP Price      = P - Stop Distance * tp_ratio
```

Why 2:1? The default TP ratio of 2.0 means the take-profit is twice as far from entry as the stop-loss. This means the strategy needs only a 33.3% win rate to break even (ignoring commissions): a 2:1 reward-to-risk means two losses and one win nets zero. At a 40% win rate with 2:1 RR, the strategy is profitable even with more losing trades than winning ones.

Higher TP ratios (3:1, 4:1) increase the reward per winning trade but reduce win rate (the TP is further away, harder to reach). Lower TP ratios (1:1, 1.5:1) increase win rate but require a higher win rate to be profitable overall.

7.5 Commission & Slippage

```
Total Cost per Trade = (commission_pct / 100) + (slippage_pct / 100)

Applied on ENTRY: entry_price_adj  = entry_price  * (1 + direction * total_cost)
Applied on EXIT:  exit_price_adj   = exit_price   * (1 - direction * total_cost)

Net Trade Return = (exit_price_adj - entry_price_adj) / entry_price_adj * direction
```

Both commission and slippage are expressed as percentages of the trade value and are applied symmetrically on entry and exit. The default values are **0.1% commission** and **0.05% slippage**, representing a realistic estimate for a liquid equity or ETF on a modern retail broker.

Commission and slippage are applied to every trade, both on entry and on exit. This means the round-trip cost is approximately 2 × (commission + slippage). For the default settings: round-trip cost ≈ 0.3% per trade.

At high trade frequencies (many short-duration trades), commission and slippage can significantly erode returns. A strategy that appears profitable gross may be unprofitable net of costs. Always inspect the trade count alongside the net returns when evaluating a strategy.

7.6 Signal Re-Entry — The Pending Entry Mechanism

A subtle but important feature of the engine concerns what happens when a trade is closed mid-bar by a stop-loss or take-profit, and the signal for that same bar is still active.

The problem: Suppose a long trade is stopped out at mid-bar, but the signal at that bar is still +1 (long). Without special handling, the engine would immediately re-enter a new long position on the same bar — entering at the SL exit price and potentially entering again in the direction that just stopped you out.

The solution — pending entry: When a trade is closed by SL or TP, the engine sets a `pending_entry` flag. On the *next* bar, if the signal is still in the same direction as the pending entry, a new trade opens at that next bar's close price. This ensures:

- No immediate re-entry on the same bar as the exit.
- Re-entry is still possible on the next bar if the signal persists.
- The engine does not miss persistent signals just because a single trade hit its stop.

```
if sl_hit or tp_hit:
    close_current_trade(exit_reason='SL' or 'TP')
    pending_entry = current_signal   # remember the signal direction

# Next bar:
if pending_entry != 0 and signal[next_bar] == pending_entry:
    open_new_trade(direction=pending_entry)
    pending_entry = 0
```

The pending entry mechanism prevents both double-trading on a single bar and signal-loss after a stop-out. It is active by default and cannot be disabled — it is a correctness feature of the simulation, not an optional strategy feature.

8 Performance Metrics

The backtest engine computes the following metrics from the equity curve and trade log. All metrics are displayed in the Results tab and included in the downloadable CSV export.

Total Return (%)

The overall percentage gain or loss over the entire backtest period, measured from the equity curve.

```
Total Return = (Final Equity - Initial Equity) / Initial Equity * 100
```

This is the raw cumulative return. It does not account for time — a 50% return over 10 years is very different from 50% over 1 year. For time-adjusted comparisons, use Annualised Return.

Annualised Return (%)

The compound annual growth rate (CAGR) — what annual return would compound to produce the observed total return over the backtest period.

```
n_years = n_bars / annualisation_factor

Annualised Return = ((1 + Total Return / 100) ^ (1 / n_years) - 1) * 100
```

The annualisation factor depends on the interval (252 for daily, 252×24/4 for 4h, etc.). This metric allows fair comparison between backtests of different lengths.

Max Drawdown (%)

The largest peak-to-trough decline in the equity curve over the entire backtest period.

```
Rolling Peak[t] = max(Equity[0 : t+1])
Drawdown[t]     = (Equity[t] - Rolling Peak[t]) / Rolling Peak[t] * 100
Max Drawdown    = min(Drawdown[t] for all t)
```

Max Drawdown is expressed as a negative percentage. It is the most commonly used risk metric in systematic trading. A strategy with a 40% annualised return but a 60% max drawdown may be psychologically impossible to trade — most investors would abandon it during the drawdown before it recovers.

Sharpe Ratio

The risk-adjusted return — annualised excess return divided by annualised volatility of returns. Measures return per unit of total risk.

```
Sharpe Ratio = (Annualised Return - Risk Free Rate) / (std(returns) * sqrt(ann_factor))

Risk Free Rate = 0 (default, can be changed in settings)
```

The Sharpe Ratio is the most widely used performance metric in finance. A Sharpe above 1.0 is generally considered good; above 2.0 is excellent; above 3.0 is exceptional (and rare outside of very high-frequency strategies). Below 0 means the strategy underperforms the risk-free rate on a risk-adjusted basis.

Limitation: The Sharpe Ratio penalises all volatility equally — including upside volatility (large gains). A strategy with large positive outliers will be penalised. The Sortino Ratio addresses this.

Sortino Ratio

Like the Sharpe Ratio, but uses only the downside deviation (standard deviation of negative returns) rather than total standard deviation.

```
Downside Returns = returns where returns < 0
Downside Std     = std(Downside Returns) * sqrt(ann_factor)

Sortino Ratio = Annualised Return / Downside Std
```

The Sortino Ratio is more appropriate than Sharpe for strategies with asymmetric return distributions — such as trend-following strategies that have many small losses and occasional large wins. A Sortino Ratio greater than 1.5–2.0 indicates good risk-adjusted performance on the downside.

Calmar Ratio

The ratio of annualised return to maximum drawdown. Measures return per unit of worst-case drawdown risk.

```
Calmar Ratio = Annualised Return (%) / |Max Drawdown (%)|
```

A Calmar Ratio above 0.5 is considered acceptable; above 1.0 is good; above 2.0 is excellent. A strategy with a 20% annualised return and a 10% max drawdown has a Calmar of 2.0.

Win Rate (%)

The percentage of completed trades that resulted in a positive return (after commission and slippage).

```
Win Rate = (Number of Winning Trades / Total Completed Trades) * 100
```

Win rate alone is not a meaningful metric without the average win/loss ratio. A strategy with a 30% win rate can be highly profitable if the average win is 3× the average loss. Conversely, a strategy with a 70% win rate can be unprofitable if the losses are much larger than the wins.

Long Win Rate / Short Win Rate (%)

Win rate broken down separately for long trades and short trades. This reveals directional bias — whether the strategy performs better on one side of the market.

```
Long Win Rate  = Winning long trades  / Total long trades  * 100
Short Win Rate = Winning short trades / Total short trades * 100
```

A strategy with a high long win rate but low short win rate may benefit from being restricted to long-only mode. These metrics are displayed in the metrics table and can guide parameter refinement.

Exposure (%)

The percentage of bars where the strategy held an active position (long or short). Measures how much of the available time was spent in the market.

```
Exposure = (Bars with non-zero position / Total bars) * 100
```

High exposure (close to 100%) means the strategy is almost always in the market — it never steps aside. Low exposure (e.g. 20–30%) means it trades selectively. For trend-following strategies, exposure often correlates with the holding period; for mean-reversion, it correlates with how often extreme readings occur.

Number of Trades

The total count of completed round-trip trades (entry + exit pairs) over the backtest period.

A low trade count (fewer than 30–50) makes all other statistics unreliable due to small sample size — a few lucky or unlucky trades can dominate the metrics. The Parameter Sweep section uses a minimum trade count filter (default 30) to exclude under-sampled parameter combinations from the heatmap.

9 Trade Log Columns

Every completed trade is recorded in the trade log, which can be viewed in the Trade Log tab and downloaded as a CSV. Each row represents one completed round-trip trade (from entry to exit).

Column	Description
Entry Date	The timestamp of the bar at which the trade was entered. This is the bar whose close price was used as the entry price.
Exit Date	The timestamp of the bar at which the trade was closed. For SL/TP exits, this is the bar that touched the level; for signal/fixed-bar exits, this is the bar the exit condition was met.
Entry Price	The close price of the entry bar (before commission and slippage adjustment). This is the price at which the position was theoretically opened.
SL Price	The stop-loss price set at entry. *NaN* if SL was not enabled for this trade. Calculated as `Entry Price ± atr_multiplier × ATR` depending on direction.
TP Price	The take-profit price set at entry. *NaN* if TP was not enabled. Calculated as `Entry Price ± Stop Distance × tp_ratio`.
Exit Price	The actual price at which the trade was closed. For SL exits, this is the SL price. For TP exits, this is the TP price. For signal/bar exits, this is the close price of the exit bar.
Exit Reason	One of: `SL` (stop-loss hit), `TP` (take-profit hit), `Signal` (signal reversed or turned neutral), `MaxBars` (maximum bars held reached), `EndOfData` (backtest period ended with trade open).
Direction	`Long` or `Short`. The direction of the trade.
Return (%)	The net percentage return of the trade after commission and slippage. A positive value is a winning trade; negative is a losing trade. $\text{Return} = (\text{Exit Price}_{adj} - \text{Entry Price}_{adj}) / \text{Entry Price}_{adj} \times \text{direction} \times 100$
Bars Held	The number of bars from entry to exit (inclusive of the entry bar). One bar on a daily chart = one trading day; one bar on a 1h chart = one hour.
MAE (%)	Maximum Adverse Excursion — the worst intra-trade loss as a percentage of entry price. Measures the deepest the trade went against you at any point during its lifetime. MAE is always a negative number (or zero if the trade never went against you).
MFE (%)	Maximum Favourable Excursion — the best intra-trade gain as a percentage of entry price. Measures the most the trade moved in your favour at any point during its lifetime. MFE is always a positive number (or zero if the trade never moved in your favour).

Using MAE and MFE Together

MAE and MFE are diagnostic tools for understanding trade behaviour beyond just the final outcome:

- A trade with a large MFE but a small final Return indicates that the exit strategy is giving back too much profit — the trade went well but exited too late. A tighter TP or earlier signal exit might capture more of the MFE.
- A trade with a small MAE that resulted in a loss (hit SL) indicates the stop was placed reasonably — the trade immediately went against the entry and the stop did its job.
- A trade with a large MAE that survived and ultimately became a winner indicates the stop was wide enough to give the trade room, but raises the question of whether a tighter stop would have improved the risk-adjusted return.
- Plotting MAE vs final Return across all trades reveals whether the stop-loss level is calibrated correctly: if most losing trades have MAE near their SL (small), the stops are being hit cleanly; if losing trades have large MAEs that exceed the SL, there may be a slippage or gapping issue.

10 Parameter Sweep

The Parameter Sweep (available in the Trend/Momentum and Mean Reversion builders) runs a grid search over two selected parameters simultaneously and visualises the results as an interactive heatmap. This allows systematic optimisation and stability testing without manual trial-and-error.

How It Works

You select two parameters to sweep (e.g. fast period and slow period for an SMA Crossover), define the range and step size for each, and click Run Sweep. The engine runs a full backtest for every combination of parameter values and records the chosen metric (e.g. Sharpe Ratio, Total Return) for each.

All other parameters remain fixed at their current values during the sweep. Only the two selected parameters vary.

The Linspace Grid

The parameter ranges are divided into evenly spaced values using numpy's linspace:

```
param_values = np.linspace(min_value, max_value, n_steps)

# Example: fast period from 5 to 30, 6 steps
# → [5, 10, 15, 20, 25, 30]
```

The number of steps controls the resolution of the sweep. More steps produce a finer grid but take proportionally longer to run. The total number of backtests is `n_steps_param1 × n_steps_param2`.

Minimum Trades Filter

Any parameter combination that produces fewer than the minimum trade count threshold (default 30) is excluded from the heatmap — its cell is shown as grey/NaN. This prevents the display of misleadingly high metrics from combinations that traded only 2–3 times.

```
if trade_count < min_trades:
    metric_value = NaN  # greyed out on heatmap
```

The Heatmap

Results are displayed as an interactive Plotly heatmap with:

- X-axis: values of the first selected parameter.
- Y-axis: values of the second selected parameter.

- Cell colour: value of the selected metric (e.g. Sharpe Ratio). Green = good, red = poor (for return/Sharpe metrics); inverted for drawdown.
- Hover tooltip: exact metric value and trade count for each cell.
- The current parameter values (from the main builder sliders) are highlighted with a marker on the heatmap, showing where the current settings fall within the explored space.

Limitations and Warnings

- **In-sample optimisation bias:** The parameter sweep is performed on the same data used for the backtest. Selecting the best-performing parameters from the sweep and reporting those results as the strategy's performance is in-sample overfitting. The swept results should be used for understanding the parameter sensitivity landscape, not for selecting a "best" parameter to then test on the same data.
- **Parameter stability is more important than the peak:** A cell with Sharpe=1.8 surrounded by cells with Sharpe=1.5–1.7 is more likely to be a robust combination than a cell with Sharpe=2.5 surrounded by cells with Sharpe=0.5. Look for smooth, stable regions on the heatmap rather than isolated peaks.
- **Computation time:** Large grids (e.g. 20×20 = 400 backtests) may take several seconds on intraday data with thousands of bars. Start with coarser grids (6×6 or 8×8) to identify the promising region, then refine.
- **The sweep does not optimise all parameters simultaneously.** Only two parameters vary at a time. The remaining parameters are held fixed, which means the optimal combination found by the sweep may not be globally optimal across all parameters.
